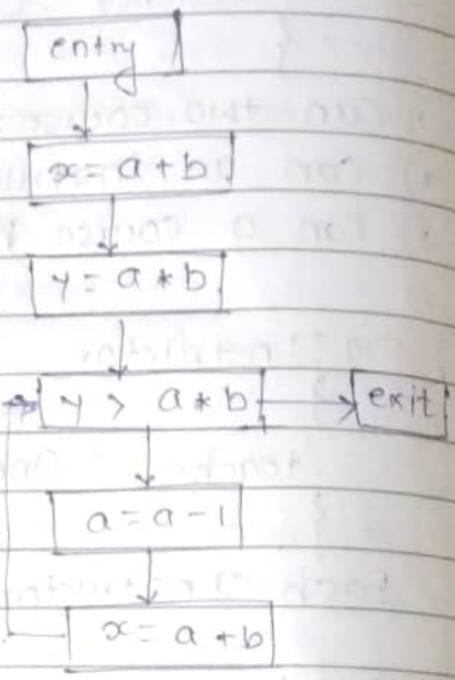
1. * Available Expression Analysis.

Q1.

```
x = a+b
y = a*b.
while (y > a+b)
  {
    a = a-1;
    x = a+b;
  }
```



1. Direction Forward
2. Domain
3. Meet Operator ('∩')
4. Transfer function :

5. $AE_{exit}(S) = (AE_{entry}(S) \setminus kill(S)) \cup Gen(S)$

6. Boundary condition. = ϕ.

state	Gen { }	kill { }
1	{a+b}	ϕ
2.	{a*b}	ϕ
3.	{a+b}	ϕ
4.	ϕ	{a+b, a*b}
5.	{a+b}	ϕ

$AE_{entry}(1) = \phi$

$AE_{entry}(2) = AE_{exit}(1) \Rightarrow \{a+b\}$

$AE_{entry}(3) = AE_{exit}(2) \cap AE_{exit}(5) \Rightarrow \{a+b, a+b\} \cap \phi$

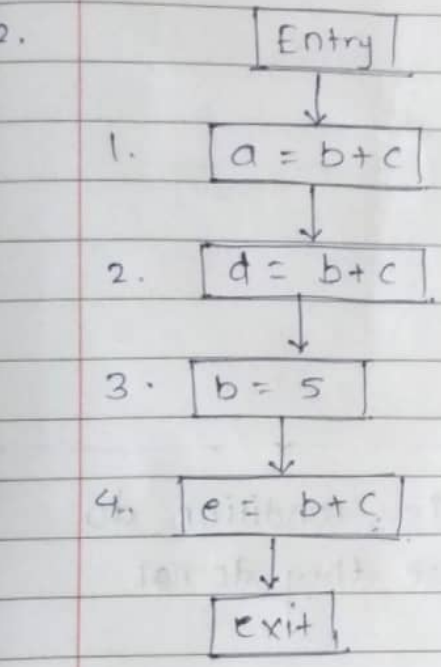
$AE_{entry}(4) = AE_{exit}(3) \Rightarrow \{a+b\}$

$AE_{entry}(5) = AE_{exit}(4) \Rightarrow \phi$

GEN contains expressions that are computed & available after the statement.

AEexit (1) = $(\phi \setminus \phi) \cup \{a+b\} = \{a+b\}$
AEexit (2) = $(\{a+b\} \setminus \phi) \cup \{a+b\} = \{a+b, a+b\}$
AEexit (3) = $\{a+b\}$
AEexit (4) = $\{a+b\} \setminus \{a+b, a+b\} \cup \phi = \phi$.
AEexit (5) = $(\phi \setminus \{a+b\}) \cup \phi = \{a+b\}$.

Q2.

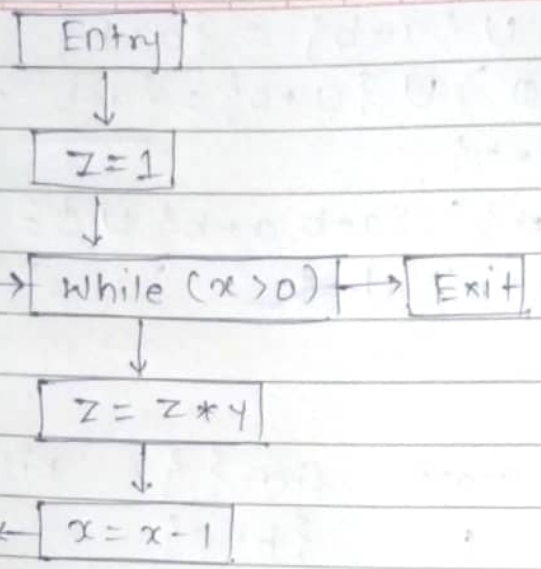


state	Gen { }	Kill { }
1.	{b+c}	ϕ
2	{b+c}	ϕ
3.	ϕ	{b+c}
4.	{b+c}	ϕ.

AEentry (1) = ϕ
AEentry (2) = {b+c}
AEentry (3) = {b+c}
AEentry (4) = ϕ

AEexit (1) = $(\phi \setminus \phi) \cup \{a+b\}) = \{b+c\}$
AEexit (2) = $(\{b+c\} \setminus \phi \cup \{b+c\}) = \{b+c\}$
AEexit (3) = $(\{b+c\} \setminus \{b+c\} \cup \phi) = \phi$
AEexit (4) = $(\phi \setminus \phi) \cup \{b+c\} = \{b+c\}$.

Q8.



State	Gen { }	Kill { }
1.	{1}	ϕ
2.		

* In available expression analysis, condition do not generate expression, because they do not store or compute expression.

Reaching Defination Analysis

Q4.
```
x = 10
y = 5
while (x > 1)
{
  y = x * y
  x = x - 1
}
```

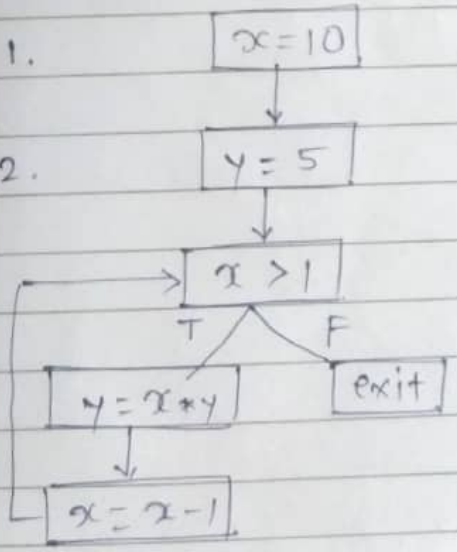
1. Domain (tupple) : (V, S)
   Assignment → Variable, statement line No.
2. Foward Direction.
3. Meet operator ('U')
4. Transfer Function.

$$RD_{exit}(S) = (RD_{entry}(S) \setminus Kill(S)) \cup gen(S)$$

5. $gen(S) = (V, S)$ otherwise ϕ
   $Kill(S)$ = if S is ~~ag~~ assignment $(V, S') \forall S'$
   otherwise ϕ.
6. Boundary Condition = Undefined variables.

$RD_{entry}(Entry\ B/N) = \{(V, ?)\}$

Q5.



State	gen(S)	Kill (S)
1.	{(x, 1)}	{(x,1), (x,?), (x,5)}
2.	{(y,2)}	{(y,2), (y,?), (y,4)}
3.	ϕ	ϕ
4.	{(y,4)}	{(y,4), (y,?), (y,2)}
5.	{(x,5)}	{(x,5), (x,?), (x,1)}.

RDentry(1) = {(x, ?) (y, ?)}.

RDentry(2)

RDentry(3)

RDentry(4)

RDentry(5)

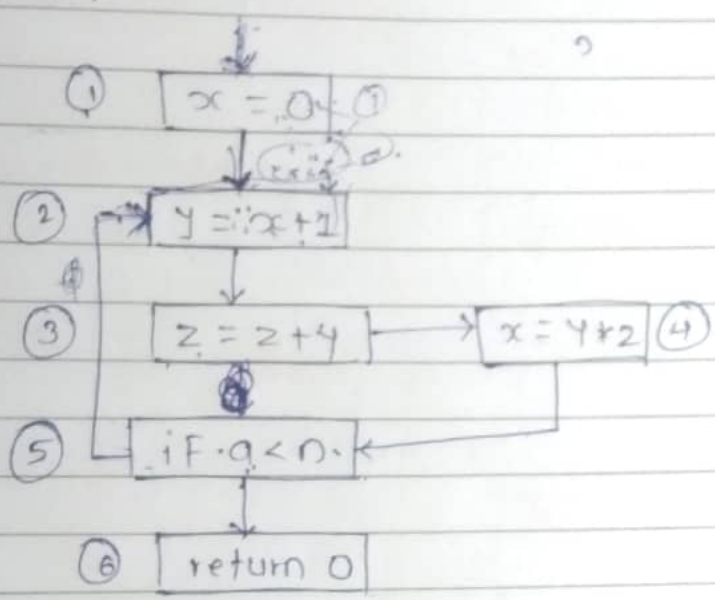
13/03/26

PAGE NO.

DATE : / /

Q.

1. $x = 0$
2. $y = x + 1$
3. $z = z + y$
4. $x = y * 2$
5. if $a < n$ goto (2),
   return 0.



State	Gen { }	Kill { }
1.	{(x, 1)}	{(x, ?), (x, 1), (x, 4)}
2.	{(y, 2)}	{(y, ?), (y, 2)}
3.	{(z, 3)}	{(z, ?), (z, 3)}
4.	{(x, 4)}	{(x, ?), (x, 1), (x, ?)}
5.	ϕ	ϕ
6.	ϕ	ϕ

RDentry (1) = {(x, ?), (y, ?), (z, ?)}
RDentry (2) = RDexit (1) =
RDentry (3) = RDexit (2) =
RDentry (4) = RDexit (3) =
RDentry (5) = RDexit (4) =
RDentry (6) = RDexit (5) =

PAGE NO.
DATE :

$RDexit(s) = (RDentry(s) \setminus kill(s)) \cup gen(s)$

$RDexit(1) = \{(x,?),(y,?),(z,?)\} \setminus \{(x,?),(y,?)$
$(z,?)\} \cup (x,1)$

13/03/26.

PAGE NO
DATE

Very Busy Expression Analysis.

(works on Future values)

Q2.

```
Fun sum (int list [], int a, int b, int c)
    {
    For (i=0; i<cnt; i++)
        {
            list [i] = a+b+c;
        }
    }
```

1. Domain - exp.
2. Direction - backward
3. Meet Operation - `n` (intersection)
4. Transfer function.

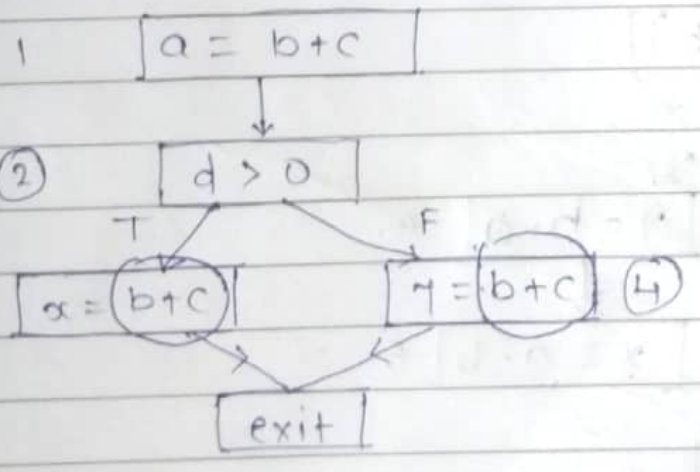
* hoisting condition.

① $VBexit(s) = VBentry(s)$

② $VBentry(s) = (VBexit(s) \setminus kill(s)) \cup gen(s)$

5. Boundary condition : ϕ at exist $\therefore VBexit = \phi$

Q1.



State	gen(s)	kill(s)
1.	{b+c}	ϕ
2.	ϕ	ϕ
3.	{b+c}	ϕ
4.	{b+c}	ϕ

$VBexit(1) = VBentry(2) = \{b+c\}$

$VBexit(2) = VBentry(3) \cap VBentry(4) = \{b+c\}$

$VBexit(3) = VBentry(4) \quad \phi$

$VBexit(4) = \phi$

$VBentry(3) = \{VBexit(3) \setminus kill(3)\} \cup gen(3)$

$= \{\phi \setminus \phi\} \cup \{b+c\}$

$= \{b+c\}$

$VBentry(4) = \{\phi \setminus \phi\} \cup \{b+c\}$

$= \{b+c\}$

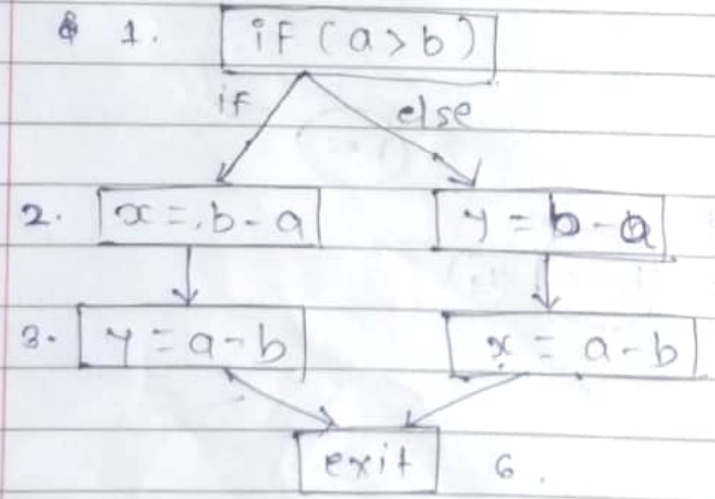
$VBentry(2) = (\{b+c\} \setminus \phi) \cup \phi$

$= \{b+c\}$

$VBentry(1) = (\{b+c\} \setminus \phi) \cup \{b+c\}$

$= \{b+c\}.$

Q2.

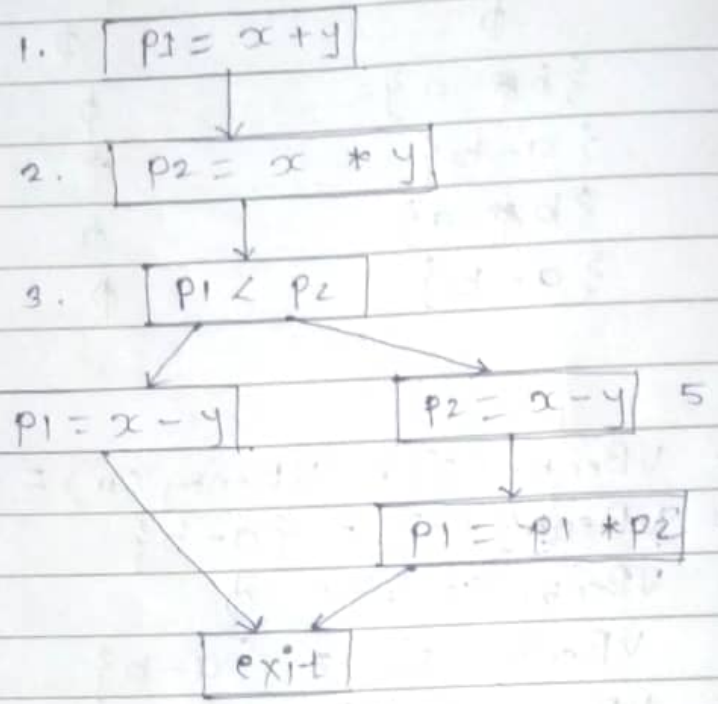


State	gen(s)	kill(s)
1.	ϕ	ϕ
2.	$\{b-a\}$	ϕ
3.	$\{a-b\}$	ϕ
4.	$\{b-a\}$	ϕ
5.	$\{a-b\}$	ϕ

$VBexit(1) = VBentry(2) \cap VBentry(4) = \{b-a\}, \{a-b\}$
$VBexit(2) = VBentry(3) = \{a-b\}$
$VBexit(3) = VBentry(exit) = \phi$
$VBexit(4) = VBentry(5) = \{a-b\}$
$VBexit(5) = VBentry(exit) = \phi$

$VBentry(1) = (\{b-a\}, \{a-b\} \setminus \phi) \cup \phi = \{b-a, a-b\}$.
$VBentry(2) = (\{a-b\} \setminus \phi) \cup \{b-a\} = \{a-b\}, \{b-a\}$
$VBentry(3) = (\phi \setminus \phi) \cup \{a-b\} = \{a-b\}$
$VBentry(4) = (\{a-b\} \setminus \{b-a\}) \cup \{b-a\} = \{a-b\}, \{b-a\}$
$VBentry(5) = (\phi \setminus \{a-b\}) \cup \{a-b\} = \{a-b\}$

Q3.
```
P1 = x + y
P2 = x + y
IF (P1 < P2)
   P1 = x - y
else
   P2 = x - y
P1 = P1 * P2
```



State	gen(s)	kill(s)
1.	ϕ	ϕ
2.	$\{x+y\}$	ϕ
3.	$\{x*y\}$	ϕ
4.	$\{x-y\}$	ϕ
5.	$\{x-y\}$	
6.	$\{p1*p2\}$	

{Gen, Kill}

When we use (U) - May analysis

19/03/2026

Live Variable Analysis

Live → Redefined

chain

Use defination ud change

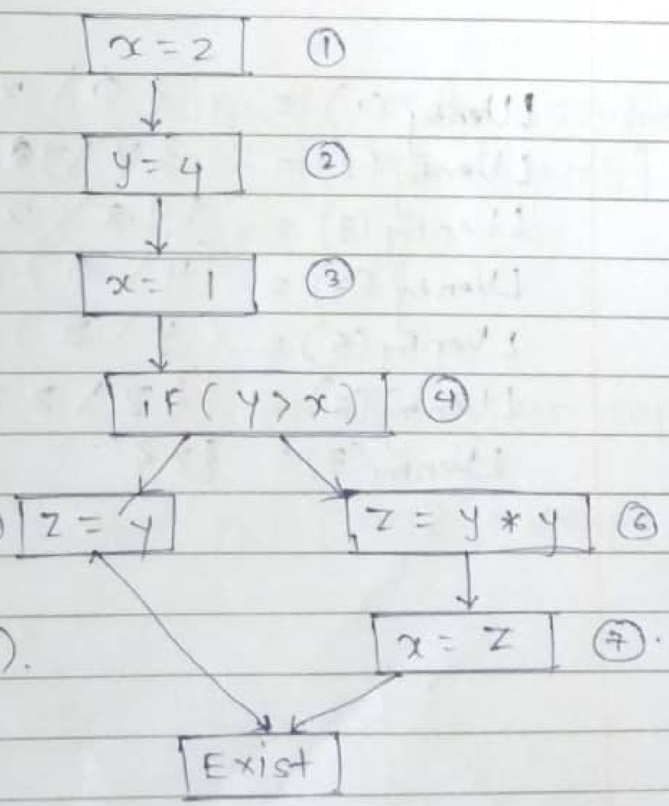
defination used du change

1. Domain → set of variables
2. Direction → Backward Analysis
3. Meet operation → 'U' (union)
4. May Analysis(U) Must Analysis (∩)
5. Boundary Condition : $LV_{exist}(s) = \phi$
6. Transfer function

$$LV_{entry} = (LV_{exit}(s) \setminus kill(s)) \cup gen(s)$$

e.g ①

```
x = 2;
y = 4;
x = 1;
if (y > x)
  { z = y }
else
  {
  z = y * y;
    x = z;
  }
```



State	gen(s)	kill(s)
1.	ϕ	{x}
2.	ϕ	{y}
3.	ϕ	{x}
4.	{y, x}	{ϕ}
5.	{y}	{z}
6.	{y}	{z}
7.	{z}	{x}

LVentry(5) = (LVexist(5) \ KILL(5)) ∪

PAGE NO.
DATE :

LVentry(1) = LVentry(2)
LVentry(2) = LVentry(3)
LVentry(3) = LVentry(4)
LVentry(4) = LVentry(5) ∪ LVentry(6)
LVentry(5) = LVentry(6)
LVentry(6) = LVentry(7)
LVentry(7) = ϕ

LVexist(1) = LVentry(2) = ϕ
LVexist(2) = LVentry(3) = {y}
LVexist(3) = LVentry(4) = {y,z}
LVexist(4) = LVentry(5) = {y}
LVexist(5) = LVentry(6) = {y}
LVexist(6) = LVentry(7) = {z}
LVexist(7) = ϕ

LVentry(1) = {ϕ \ x} ∪ ϕ = ϕ
LVentry(2) = {y \ y} ∪ {ϕ} = ϕ
LVentry(3) = {y, x \ x} ∪ ϕ = {y}
LVentry(4) = (y \ ϕ) ∪ {y,z} = {y,x}
LVentry(5) = (y \ z) ∪ {y} = {y}
LVentry(6) = ({z \ z}) ∪ {y} = {y}
LVentry(7) = {z}

PAGE NO.
DATE : / /

1. Rice's Theorem
2. Under Approximation
3. Over Approximation.

* Tainy Language:

i) Variable, Nested Loops, Functions, Objects

Basic ~~oper~~ Expression.

1. - int : 1 | 0 | -1 | -2
2. - id : x | y | z | a | b
3. - exp : int | id | exp + exp | exp - exp | exp * exp | exp \ exp | exp > exp. | exp == exp | (exp) | input
4. - Comparision : 0 - false
   1 - True
5. - Statements : id = exp ; | Output | statmt : statmt
6. - | iF (exp) {state} | [else {state}] ? | While (exp) {state} |
7. functions : ~~Fun~~ id (id, id, ...) [var, id ...] ? {state return exp}]
   ~~exp~~ → ...
8. pointer : exp → ... | alloc exp | ↑id | * exp | NULL.
9. Assignment statement : exp | id := exp.

Operational Semantic

Transition system (change in state)
- Configuration
  $\langle S, s \rangle$
  - S: statement which is going to executed
  - s: change in state (terminal state)

$[[\ ]]_v$

~~bbb~~ Consider a value of variable, if it is integer or Boolean (T/F).

example:

```
short ()
{
 var x, y, z;
  x = *p;
  y = alloc x;
  *y = x;
  z = x
  z = *y
  }
 return z;
```

$[[short]] = () \rightarrow [[z]]$
$[[*p]] = int$
$[[x]] = [[*p]]$
$[[alloc]] = [[x]]$
$[[y]] = [[x]]$
$[[z]] = [[*y]]$

Natural Operational Semantic (NOS) (Big St

1) Assignment

$\langle x := a, s \rangle \rightarrow s[x \rightarrow A[[a]]_s]$

2) Skip

$\langle skip, s \rangle \rightarrow s$

3) Composition.

~~$\langle S_1 ; S_2 , s \rangle \rightarrow s$~~

$$\frac{\langle S_1, s \rangle \rightarrow s' \quad \langle S_2, s' \rangle \rightarrow s''}{\langle S_1 ; S_2, s \rangle \rightarrow s''}$$

4) if statement Rule.

$[[\text{if}]]_{tf}$

$$\frac{\langle S_1, s \rangle \rightarrow s'}{\langle \text{IF } B \text{ then } S_1 \text{ else } S_2, s \rangle \rightarrow s'}$$

5) Looping structure.

$$\frac{\langle S, s \rangle \rightarrow s' \quad \langle \quad , s' \rangle \rightarrow s''}{\langle \text{while } B \text{ do } S, s \rangle \rightarrow s''}$$

24/03/2025

```
x := 1      s := 0
While (x ≤ 5)
{
   s := s + x;
   x := x + 1;
}
```

$2 \leq 5$

$s = 1 + 2 = 3$

$x = 1 + 1 = 2$

$0 = 0 + 1 = 1$

$1 = 1 + 2 = 3$

$s = 3 + 3 = 6$

$s = 6 + 4 = 10$

$s = 10 + 5 = 15$

$\langle S, [x \mapsto 1, s \mapsto 0] \rangle \longrightarrow [x \mapsto 6, s \mapsto 15]\, s'$

i/p $\longrightarrow$ o/p.

$S \qquad S'$

$[x := 1, s := 0]_s$.

$(x \leq 5) \rightarrow [[x]]_s = [x \mapsto 64]$

$\langle s = s + x, [x \mapsto 1, s \mapsto 0]_s \rangle$

$\longrightarrow [x \mapsto 1, s \mapsto 1]\, s'$

$\langle x := x + 1, s' \rangle \longrightarrow [x \mapsto 2, s \mapsto 1]\, s''$

Composite Derviation Tree.

1) Complete function
2) Partial function

Q.1 $[[x = 8, y = 2, z = 0]]_s$

$\langle z := x, x := y ; y := z, s \rangle \mapsto s'$

$\langle z := x, x := y, y := z, s \rangle \longrightarrow s^3$

PAGE NO.
DATE: / /

$\langle x := y, s\rangle \to s_1' \quad \langle y := z, s_1'\rangle \to s_2$

$\langle z := x, s\rangle \to s_1 \quad \langle x := y, y := z, s_1\rangle \to s_2.$

$\langle z := x, x := y, y := z, s\rangle \to s^3$

Q2. $[[x \mapsto 0]]_s$

skip;

$x := x + 1.$

$\langle x := 0, \text{skip}, x := x + 1, s\rangle \to s'$

$$\dfrac{\dfrac{[\text{skip}, s] \to s}{\langle \text{skip}, s\rangle \to s} \quad \dfrac{s[x \mapsto 0] \to s^1[x \mapsto 1]}{\langle x := x+1, s\rangle \to s'}}{\langle \text{skip}, x := x + 1, s\rangle \to s'} \quad [\text{skip}_{ns}] \; [\text{comp}_{ns}]$$

Q3. $(2 * y) + (10 * x) = 38.$

$[[x \to 3, y \to 4]]$

$[[8 + 30]] \to s'$

$\langle (2 * y) + (10 * x) = 38, s\rangle \to s'$

Q4. if $x = 0$ then
skip,
else
$x := x + 1$

Q5. if $m > n$ then
$x := m;$
else
$y := 2 \times m.$
fi

$m = 10, \; n \to 30.$

Q1.

$$\text{[Comp]}\ \frac{\text{[Skip]}\ \dfrac{[\text{skip}, s] \to s}{\langle \text{skip}, s\rangle \to s} \quad \dfrac{[x \to 0] \to s'}{\langle x := x+1, s\rangle \to s'}}{\langle \text{skip}, x := x+1, s\rangle \to s'}$$

2. Factorial

$$\frac{\langle y := 1, s\rangle \to s_{13} \qquad T_1}{\langle y := 1 ; \text{while } \neg(x = 1) \text{ do } (y := y * x ; x := x - 1), s\rangle \to s_6}$$

$$\frac{\langle \text{while } \neg(x = 1) \text{ do } (y := y * x ; x := x - 1), s_{13}\rangle \to s_6}{(\text{True / False})}$$

T_2 : $\langle y := y * x ; x := x - 1, s_{13}\rangle \to s_{32}$

T_3 : $\langle \text{while } \neg(x = 1) \text{ do } (y := y * x ; x := x - 1), s_{32}\rangle \to s$

Principle OF program Analysis.

```
if m > n then
    x := m
else
    y := 2 * m
Fi
```

m = 10 n = 30.

$$\frac{\dfrac{S[x \to 10] \to S' \quad \langle x := m, S\rangle \to S'}{\langle \text{if } m>n \text{ then } x:=m \text{ else } y:=2*m\rangle \to S'}}{\text{b} \quad S_1 \quad S_2 \quad \langle S_2, S\rangle \to S'}$$

$$\frac{S[m \to 10]}{\langle y := 2*m, S\rangle \to S'}\ [Ass]$$

$$\frac{}{\langle m>n \text{ then } x:=m \text{ else } y:=2*m, S\rangle \to S'}\ [IF]_{FF}$$

S_1 S_2

Structural Operational Semantics.
(small step).

1. $[ass_{sos}]$ $\langle x := a, s \rangle \Rightarrow s[x \to \mathcal{A}[\![a]\!]s]$

2. $[skip_{sos}]$ $\langle skip, s \rangle \Rightarrow s$

3. $[Comp1_{sos}]$

$$\frac{\langle S_1, s \rangle \to \langle S_1', s' \rangle}{\langle S_1; S_2, s \rangle \to \langle S_1'; S_2, s' \rangle}$$

4. $[Comp2_{sos}]$

$$\frac{\langle S_1, s \rangle \to s'}{\langle S_1; S_2, s \rangle \to \langle S_2, s' \rangle}$$

5. $[IF^{tt}_{sos}]$ $\langle$ if b then S_1 else S_2, s $\rangle \to \langle S_1, s \rangle$ if $\mathcal{B}$

6. $[IF^{FF}_{sos}]$ $\langle$ if b then S_1 else S_2, s $\rangle \to \langle S_2, s \rangle$
if $\mathcal{B}[\![b]\!]_s$

7. $[While_{sos}]$

$\langle$ while b do s, s $\rangle \to$
$\langle$ if b then (s; while b do s) else skip, s $\rangle$.

B - boolean values.

PAGE NO.
DATE : / /

Denotational Semantics.

① $S_{ds}[[x := a]]s = s[x \rightarrow A[[a]]s]$.

② $S_{ds}[[skip]] = id$. (identity of given state).

③ $S_{ds}[[S_1 ; S_2]] = S_{ds}[[S_2]] \circ S_{ds}[[S_1]]$.

④ $S_{ds}[[\text{if } b \text{ then } S_1 \text{ else } S_2]] = cond(B[[b]], S_{ds}[[S_1]], S_{ds}[[S_2]])$

⑤ $S_{ds}[[\text{while } b \text{ do } S]] = FIX\ F$

where $F\ g = cond(B[[b]], g \circ S_{ds}[[S]], id)$.

F - Functional composition:
means every mathematical object treated as mathematical functions.

~~Denotational~~
- combining two or more functions such that the output of one function becomes the input of another.

$F : A \rightarrow B$
$g : B \rightarrow C$

$(g \circ F) : A \rightarrow C$
$(g \circ F)(x) = g(F(x))$.